

Цель работы:получить навыки создания простейших ассемблерных программ с использованием пакета `masm32` и научиться пользоваться отладчиком `x32dbg`.

1. Ознакомиться со средой `x32dbg` и компилятором `masm32`.
2. Создать и скомпилировать программу в соответствии с вариантом задания. В программу включить комментарии с описанием, что делает каждая инструкция. Подробное описание каждой команды можно найти в приложении учебника В.И. Юрова «Assembler», начиная со стр. 511. Комментарии следует выравнивать по левому краю (как в примере)
3. С помощью отладчика определить местонахождение переменных, строк и массивов в сегменте данных, а также их размер. Составить таблицу и подробное описание ячеек сегмента данных (как в примере).
4. Выполнить пошаговую трассировку программы. Определить какие регистры, флаги и ячейки памяти изменяют свои значения в процессе выполнения команд. Обеспечить корректное завершение программы вызовом системной функции `ExitProcess` с кодом завершения 0. Если в сегменте данных есть строки, то вывести её в консоль. Трассировку требуется выполнить до команды «`call ExitProcess`» включительно. Составить для каждой инструкции таблицу трассировки (как в примере).
5. Сделать выводы о проделанной работе.

Задание №2. Создать и скомпилировать программу в соответствии с вариантом задания. В программу включить комментарии с описанием, что делает каждая инструкция. Подробное описание каждой команды можно найти в приложении учебника В.И. Юрова «Assembler», начиная со стр. 511. Комментарии следует выравнивать по левому краю (как в примере)

```
1  .686
2  .model flat, stdcall
3  option casemap: none
4  include windows.inc
5  include kernel32.inc
6  include msvcrt.inc
7  includelib kernel32.lib
8  includelib msvcrt.lib
9
10 .DATA
11     printf DB 13, 10, "Result: %d", 0    ; CR, LF, строка формата для printf
12     a      DW 5                          ; a = 5 (2 байта)
13     b      DD 60                         ; b = 60 (4 байта)
14     r      DD ?                          ; r - результат (4 байта)
15     dqq    DQ 171.233                   ; dqq = 171.233 (8 байт, double)
16
17 .CODE
18 START:
19     XOR EAX, EAX                        ; EAX = 0
20     MOV AX, a                          ; AX = 5 (a - слово)
21     MOV EBX, b                          ; EBX = 60
22     ADD EAX, EBX                        ; EAX = 5 + 60 = 65
23     MOV r, EAX                          ; r = 65
24
25     ; Вывод результата
26     push r                              ; второй аргумент: значение r
27     push offset printf                  ; первый аргумент: адрес строки формата
28     call crt_printf                     ; вызов printf("Result: %d", r)
29     add esp, 8                          ; очистка стека (2 аргумента по 4 байта)
30
31     ; Задержка, чтобы увидеть результат
32     call crt__getch                     ; ожидание нажатия клавиши (getch)
33
34     ;Завершение программы
35     push 0
36     call ExitProcess
37 END START
```

Задание №3. С помощью отладчика определить местонахождение переменных, строк и массивов в сегменте данных, а также их размер. Составить таблицу и подробное описание ячеек сегмента данных (как в примере).

Таблица 1: Переменные сегмента данных

Переменная	Нач. адрес	Кон. адрес	Байт	Описание
_	00403000	00403001	2	символы возврата каретки (13), перевода строки (10)
printf	00403002	0040300B	11	форматная строка для функции printf
_	0040300C	0040300C	1	Окончание строки (0)
a	0040300D	0040300E	2	2-байтовая переменная со значением 5
b	0040300F	00403012	4	4-байтовая переменная со значением 60
r	00403013	00403016	4	неинициализированная 4-байтовая переменная
dqq	00403017	0040301E	8	8-байтовая вещественная переменная со значением 171.233
Общий размер сегмента данных:			32	

Адрес	Шестнадцатеричное																ASCII
00403000	0D	0A	52	65	73	75	6C	74	3A	20	25	64	00	05	00	3C	..Result: %d...<
00403010	00	00	00	00	00	00	00	FA	7E	6A	BC	74	67	65	40	00	 j4tge@.
00403020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00403030	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

Таблица 2: Шестнадцатеричный дамп памяти с выделением переменных

Адрес	Шестнадцатеричное																ASCII
00403000	0D	0A	52	65	73	75	6C	74	3A	20	25	64	00	05	00	3C	..Result: %d...<
00403010	00	00	00	00	00	00	00	FA	7E	6A	BC	74	67	65	40	00	 j4tge@.
00403020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00403030	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

Задание 4. Выполнить пошаговую трассировку программы. Определить какие регистры, флаги и ячейки памяти изменяют свои значения в процессе выполнения команд. Обеспечить корректное завершение программы вызовом системной функции ExitProcess с кодом завершения 0. Если в сегменте данных есть строки, то вывести её в консоль. Трассировку требуется выполнить до команды «call ExitProcess» включительно. Составить для каждой инструкции таблицу трассировки (как в примере).

Таблица 3: Исходное состояние регистров

EAX=	0019FFCC	EBX=	0035B000	ECX=	00401000	EDX=	00401000
ESP=	0019FF74	EBP=	0019FF80	ESI=	00401000	EDI=	00401000
EIP=	00401000						
ZF=	1	PF=	1	AF=	0		
OF=	0	SF=	0	DF=	0		
CF=	0	TF=	0	IF=	1		

Таблица 4: Состояние регистров после команды `xor eax, eax`

xor eax, eax					КОП:	33C0	
EAX=	00000000	EBX=	0035B000	ECX=	00401000	EDX=	00401000
ESP=	0019FF74	EBP=	0019FF80	ESI=	00401000	EDI=	00401000
EIP=	00401002						
ZF=	1	PF=	1	AF=	0		
OF=	0	SF=	0	DF=	0		
CF=	0	TF=	0	IF=	1		
Обнуляет EAX. EIP увеличивается на 2.							

Таблица 5: Состояние регистров после команды `mov ax, word ptr ds:[40300D]`

mov ax,word ptr ds:[40300D]					КОП:	66:A1 0D304000	
EAX=	00000005	EBX=	0035B000	ECX=	00401000	EDX=	00401000
ESP=	0019FF74	EBP=	0019FF80	ESI=	00401000	EDI=	00401000
EIP=	00401008						
ZF=	1	PF=	1	AF=	0		
OF=	0	SF=	0	DF=	0		
CF=	0	TF=	0	IF=	1		
Загружает слово из памяти по адресу 40300D в младшую часть EAX (AX = 5). Флаги не меняются. EIP увеличивается на 6.							

Таблица 6: Состояние регистров после команды `mov ebx,dword ptr ds:[40300F]`

mov ebx,dword ptr ds:[40300F]					КОП:		8B1D 0F304000	
EAX=	00000005	EBX=	0000003C	ECX=	00401000	EDX=	00401000	
ESP=	0019FF74	EBP=	0019FF80	ESI=	00401000	EDI=	00401000	
EIP=	0040100E							
ZF=	1	PF=	1	AF=	0			
OF=	0	SF=	0	DF=	0			
CF=	0	TF=	0	IF=	1			
Загружает двойное слово из памяти по адресу 40300F в EBX (EBX = 60). Флаги не меняются. EIP увеличивается на 6.								

Таблица 7: Состояние регистров после команды `add eax,ebx`

add eax,ebx					КОП:	03C3	
EAX=	00000041	EBX=	0000003C	ECX=	00401000	EDX=	00401000
ESP=	0019FF74	EBP=	0019FF80	ESI=	00401000	EDI=	00401000
EIP=	00401010						
ZF=	0	PF=	1	AF=	1		
OF=	0	SF=	0	DF=	0		
CF=	0	TF=	0	IF=	1		
EAX = EAX + EBX = 5 + 60 = 65 (0x41). ZF=0 (результат не ноль), AF=1 (перенос из 3-го бита). EIP увеличивается на 2.							

Таблица 8: Состояние регистров после команды `mov dword ptr ds:[403013],eax`

mov dword ptr ds:[403013],eax					КОП:	A3 13304000	
EAX=	00000041	EBX=	0000003C	ECX=	00401000	EDX=	00401000
ESP=	0019FF74	EBP=	0019FF80	ESI=	00401000	EDI=	00401000
EIP=	00401015						
ZF=	0	PF=	1	AF=	1		
OF=	0	SF=	0	DF=	0		
CF=	0	TF=	0	IF=	1		
Записывает EAX (0x41) в память по адресу 403013 (переменная r). Флаги не меняются. EIP увеличивается на 5.							

Таблица 9: Состояние регистров после команды `push dword ptr ds:[403013]`

push dword ptr ds:[403013]					КОП:		FF35 13304000	
EAX=	00000041	EBX=	0000003C	ECX=	00401000	EDX=	00401000	
ESP=	0019FF70	EBP=	0019FF80	ESI=	00401000	EDI=	00401000	
EIP=	0040101B							
ZF=	0	PF=	1	AF=	1			
OF=	0	SF=	0	DF=	0			
CF=	0	TF=	0	IF=	1			
Помещает значение r (0x41) в стек. ESP уменьшается на 4 → 0019FF70. В памяти [0019FF70] = 41 00 00 00. EIP увеличивается на 6.								

Таблица 10: Состояние регистров после команды `push lab1.403000`

push lab1.403000					КОП:		68 00304000	
EAX=	00000041	EBX=	0000003C	ECX=	00401000	EDX=	00401000	
ESP=	0019FF6C	EBP=	0019FF80	ESI=	00401000	EDI=	00401000	
EIP=	00401020							
ZF=	0	PF=	1	AF=	1			
OF=	0	SF=	0	DF=	0			
CF=	0	TF=	0	IF=	1			
Помещает адрес строки формата (00403000) в стек. ESP уменьшается на 4 → 0019FF6C. EIP увеличивается на 5.								

Таблица 11: Состояние регистров после команды `call dword ptr ds:[<&printf>]`

call dword ptr ds:[<&printf>]					КОП:		FF15 08204000	
EAX=	0000000C	EBX=	0000003C	ECX=	BD4B3F15	EDX=	00000000	
ESP=	0019FF6C	EBP=	0019FF80	ESI=	00401000	EDI=	00401000	
EIP=	00401026							
ZF=	1	PF=	1	AF=	0			
OF=	0	SF=	0	DF=	0			
CF=	0	TF=	0	IF=	1			
Вызывает printf. Возвращает число выведенных символов: EAX = 0x0C (12). ECX и EDX перезаписаны функцией. На экран выведено: "\r\nResult: 65".								

Таблица 12: Состояние регистров после команды `add esp,8`

add esp,8					КОП:	83C4 08	
EAX=	0000000C	EBX=	0000003C	ECX=	BD4B3F15	EDX=	00000000
ESP=	0019FF74	EBP=	0019FF80	ESI=	00401000	EDI=	00401000
EIP=	00401029						
ZF=	0	PF=	1	AF=	1		
OF=	0	SF=	0	DF=	0		
CF=	0	TF=	0	IF=	1		
Очистка стека: ESP увеличивается на 8 → 0019FF74. Флаги ZF=0 (результат не ноль), AF=1. EIP увеличивается на 3.							

Таблица 13: Состояние регистров после команды `call dword ptr ds:[<_getch>]`

call dword ptr ds:[&_getch< >]					КОП:		FF15 0C204000	
EAX=	0000000D	EBX=	0000003C	ECX=	BD4B3F1D	EDX=	0019FDD0	
ESP=	0019FF74	EBP=	0019FF80	ESI=	00401000	EDI=	00401000	
EIP=	0040102F							
ZF=	1	PF=	1	AF=	0			
OF=	0	SF=	0	DF=	0			
CF=	0	TF=	0	IF=	1			
Вызывает getch. Возвращает код нажатой клавиши в EAX = 0x0D (Enter). ECX и EDX перезаписаны. Флаги изменились внутри функции.								

Таблица 14: Состояние регистров после команды `push 0`

push 0					КОП:	6A 00	
EAX=	0000000D	EBX=	0000003C	ECX=	BD4B3F1D	EDX=	0019FDD0
ESP=	0019FF70	EBP=	0019FF80	ESI=	00401000	EDI=	00401000
EIP=	00401031						
ZF=	1	PF=	1	AF=	0		
OF=	0	SF=	0	DF=	0		
CF=	0	TF=	0	IF=	1		
Помещает в стек код завершения 0. ESP уменьшается на 4 → 0019FF70. Флаги не меняются. EIP увеличивается на 2.							